Network Programming — Complete Theory Guide

Process Control · Signals · Pipes & FIFOs · Mutexes & Condition Variables — built from your professor's slides + Stevens APUE/UNP in depth

Contents

How this document is organized **3**

PART 1 — Process Control **3**
1.1 How every process is born . . . 3
1.2 `fork()` in full . . . 3
1.3 File sharing across `fork()` — her exact inheritance list . . . 4
1.4 Process termination . . . 5
1.5 Replacing the program of a process: `exec()` . . . 5
1.6 Process Control — her summary (know each line) . . . 6
1.7 The mechanism underneath — three kernel data structures (Stevens depth, explains *why* file sharing works) . . . 6
1.8 Stevens' synchronization idiom (fixes the fork ordering race, ties directly to her Exercise on signals below) . . . 6

PART 2 — Signals **7**
2.1 What signals are . . . 7
2.2 The basic (non-POSIX) interface: `signal()` . . . 7
2.3 Deficiencies of `signal()` — memorize both problems . . . 8
2.4 Sending and waiting for signals . . . 8
2.5 POSIX signals — the reliable interface . . . 8
2.6 Reentrant functions . . . 10
2.7 Signals — her summary . . . 11
2.8 Her exercises — worked answers . . . 11

PART 3 — Pipes and FIFOs **12**
3.1 Introduction . . . 12
3.2 IPC persistence — a classification she explicitly tests . . . 12
3.3 Pipes . . . 12
3.4 FIFOs . . . 13
3.5 Properties of pipes and FIFOs . . . 14
3.6 Pipes and FIFOs — her summary . . . 15

PART 4 — Mutexes and Condition Variables **15**
4.1 Introduction . . . 15
4.2 Mutexes . . . 15
4.3 Condition variables . . . 17
4.4 `pthread_mutex_lock()` internals worth stating precisely . . . 18
4.5 Her exercises for Mutexes and Condition Variables — worked answers . . . 18
4.6 Mutexes and Condition Variables — her summary . . . 18

PART 5 — Tricky concepts & likely questions, by topic **19**
5.1 Process Control — tricky points 19
5.2 Signals — tricky points 19
5.3 Pipes/FIFOs — tricky points 20
5.4 Mutexes/Condition Variables — tricky points 20

PART 6 — Consolidated Question Bank (75+ questions) **20**
Process Control 20
Signals 21
Pipes and FIFOs 21
Mutexes and Condition Variables 22
Cross-topic / integrative 22

How this document is organized

Your professor's slides give the **skeleton** — the function names, the exact programs she wants you fluent in, and the questions she likes to ask ("Q: …"). Stevens (APUE and UNP vol. 2, both of which she cites by program number) gives the **depth** she says you actually need to pass.

For every topic below: 1. **Full theory explanation** — in enough depth to answer a "explain X" essay question, not just define it. 2. **The exact programs her slides reference** (by program number where she cites one), cleaned up and annotated. 3. **Tricky concepts and likely questions specific to that topic**, including her own "Q:" prompts and exercises, verbatim where she gave them.

A final consolidated question bank sits at the end.

Scope note: her syllabus for this exam covers Process Control → Signals → Pipes/FIFOs → Mutexes & Condition Variables. Her slide deck continues into POSIX Semaphores, Shared Memory, and Socket Programming, but those come after 17/8 in her schedule — outside tomorrow's exam — so this guide stops at Condition Variables, matching what you're actually being tested on.

PART 1 — Process Control

1.1 How every process is born

Other than `init`/`swapper`/`pagedaemon` (kernel bootstrap processes), **every process in UNIX is created by `fork()`**. This is worth stating explicitly if asked "what creates a process" — the answer isn't "the OS," it's "an existing process calling `fork()`," all the way back to `init`.

1.2 `fork()` in full

```c
#include <unistd.h>
pid_t fork(void);
```

- Returns **0** in the child, the **child's PID** in the parent, **-1** on error.
- `fork()` creates a **copy of the calling process**; parent and child then execute **simultaneously** (order between them is not defined by the standard).

Copy-on-write — say this precisely, not vaguely

Her slides state it exactly: **"Data, stack and heap of a parent are not copied. The protection is set to read-only until the parent or child attempts to write, at which time a copy is made."** - This means immediately after `fork()`, parent and child's data/stack/heap pages are the **same physical memory**, marked read-only. - The **first write** by either process to a COW page triggers a page fault, the kernel duplicates just that page, and the writing process gets its own private copy from that point on. - This is a **performance optimization**, not a semantic difference — logically the child still behaves as if it got a full independent copy at the instant of `fork()`; COW just avoids the cost when most `fork()`s are immediately followed by `exec()` (which throws the copied memory away anyway).

Program 8.1 from APUE (her reference — know this exactly)

```c
int glob = 6;
char buf[] = "a write to stdout\n";

int main(void) {
    int var;
    pid_t pid;
    var = 88;

    if (write(STDOUT_FILENO, buf, sizeof(buf) - 1) != sizeof(buf) - 1)
        printf("write error");
    printf("before fork\n");

    if ((pid = fork()) > 0) {
        sleep(2);
    } else if (pid == 0) {          /* child */
        glob++;
        var++;
    } else {
        printf("Error forking\n");
    }
    printf("pid = %d, glob = %d, var = %d\n", getpid(), glob, var);
}
```

What this program is testing (this is her go-to example, be ready to trace it exactly): - `write(STDOUT_FILENO, buf, ...)` happens **before** `fork()` and is an **unbuffered** system call — it executes exactly once, no matter what happens after. You will only ever see `"a write to stdout"` printed **once**. - `printf("before fork\n")` also happens before `fork()`, but goes through **buffered stdio**. If stdout is a **terminal** (line buffered), the `\n` flushes it immediately, so you see it once. If stdout is **redirected to a file** (fully buffered), it may still be sitting in the buffer, unflushed, at the moment of `fork()` — the child inherits a **copy of that buffer**, and both parent and child flush their own copy independently at exit, so `"before fork"` can appear **twice** in the file. - After `fork()`, the **parent sleeps 2 seconds** while the **child increments `glob` and `var` immediately** — this ordering is deliberate in the program so the child's `printf` reliably appears before the parent's (the parent is forced to wait), removing the usual race, at least for *this specific* pair of print statements. - The child's final line shows `glob = 7, var = 89` (incremented copies); the parent's shows `glob = 6, var = 88` (untouched, since `glob`/`var` are independent copies after `fork()`).

1.3 File sharing across `fork()` — her exact inheritance list

When a process is created via `fork()`, the child **inherits all open file descriptors** from the parent, plus: - current working directory - environment - **file mode creation mask** (`umask`) - signal mask - (and more — controlling terminal, root directory, resource limits)

What's different in parent vs. child: the return value of `fork()`, the process ID, the parent process ID.

File sharing rule (say this precisely): *"Parent and child share the same file offset so that when one process writes to the file, the new offset is visible to the other process. Otherwise, they will overwrite each other."* This is the mechanism explained in depth in Part 1.7 below (the three-table kernel model) — the professor's slide gives you the *what*, Stevens gives you the *why*.

Her Exercise 2 (verbatim, be ready to answer this practically)

"Open a file in a program, fork and write to the file in both parent and child — use both unbuffered and buffered write and see the results." - **Unbuffered (`write()`):** each `write()` call is a direct system call, so bytes from parent and child land in the file in whatever order the kernel schedules them — sequential, no overwriting, because the shared offset advances correctly with each `write()`. - **Buffered (`printf`/`fprintf` to a `FILE*` on that fd):** each process has its **own stdio buffer** in its own memory (buffers are user-space, not shared even though the fd is) — so buffered output from parent and child can be flushed at unpredictable times relative to each other, potentially producing output that looks out of order or clumped together, **even though the underlying file offset is still correctly shared** at the moment each flush's `write()` actually happens.

1.4 Process termination

- **Normal termination:** `exit()` — implicitly called at the end of every program (i.e., falling off the end of `main()` is equivalent to calling `exit()`).
- **Abnormal termination:** `abort()` — sends `SIGABRT` to the calling process, and by default this terminates it (typically producing a core dump).
- **What happens when the parent dies first?** The child is **reparented to `init`** (or the nearest subreaper); `init` will eventually `wait()` for it.
- **What happens when the child dies first (and the parent hasn't waited)?** The child becomes a **zombie**. **Issues with zombies:** they occupy a process-table slot (finite resource — a system can run out of process table entries / hit its process limit if zombies accumulate unbounded, e.g., in a concurrent server that forks per-connection and never reaps), and they show up as `<defunct>` in `ps`, which is also an operational/monitoring nuisance.

```
pid_t wait(int *statloc);
pid_t waitpid(pid_t pid, int *statloc, int options); /* WNOHANG = non-blocking */
```

1.5 Replacing the program of a process: `exec()`

`exec()` and its variants **replace the program executing in a process** with a new one — i.e., the **data, stack, heap, and text are replaced** with those of the new program. (The professor's phrasing is worth memorizing verbatim for a definition-style question.)

Properties inherited from the calling process across `exec()` (her exact list): - pid, ppid - current working directory (pwd) - file mode creation mask - **file locks** - process signal mask - **pending signals**

```
int execl(const char *path, const char *arg, ...);
int execv(const char *path, char *const argv[]);
```

Her exec demonstration program

```
int main(int argc, char **argv)
{
    .... Initialization ....
    if ((child_pid = fork()) == -1) {
        printf("Error forking\n");
        exit(3);
    } else if (child_pid == 0) { /* Child */
        sprintf(fd_string, "%d", fd);
```

```
        execl("/home/anupama/unp/test/child", fd_string, argv[2], NULL);
    } else {                        /* Parent */
        printf("In Parent...\n");
    }
}
```

Notice this is exactly the "pass an fd as a stringified argument into the exec'd program" pattern from your lab: the fd survives `exec()` (properties inherited include file descriptors implicitly, since they're not explicitly listed as *replaced*), so the new program image can reconstruct the integer with `atoi()` and use it directly.

1.6 Process Control — her summary (know each line)

- Every process can be created only by another process in UNIX except for `init`, using `fork()`.
- Child processes created with `fork()` execute in parallel with their parents; **copy-on-write** saves the kernel from copying all of the parent's data to the child unless required.
- New programs can be executed in the context of an existing process using the family of `exec()` calls.
- **Zombie processes are created when a child dies and a parent has not waited for its exit status.**
- **A concurrent server needs to take care of zombie processes or risk running out of process space.** (This is her practical framing — connect it to the "TCP Concurrent Server" pattern: a server that `fork()`s once per incoming connection *must* reap each child, typically via a `SIGCHLD` handler, or it will eventually exhaust its process table under sustained load.)

1.7 The mechanism underneath — three kernel data structures (Stevens depth, explains *why* file sharing works)

Stevens describes three structures that together implement the file-sharing rule stated above:

1. **Per-process file descriptor table** — one per process; each entry: fd flags (`FD_CLOEXEC`) + a pointer into the file table.
2. **System-wide open file table** — shared across all processes; each entry: file status flags (read/write/append set at `open()`), the **current file offset**, and a pointer to the v-node table entry.
3. **V-node (i-node) table** — one entry per actual open file: type, size, permissions, etc.

The rule, precisely: - Two **independent `open()` calls** on the same file → **two separate file table entries**, each with **its own offset** — writing in one does not affect the other's position. - A `fork()`-inherited fd (or a `dup()`/`dup2()`'d fd) → **points to the same file table entry** as the original → **shared offset** — this is exactly the "otherwise, they will overwrite each other" the professor warns about: if instead each process had *independently* opened the file (two separate `open()` calls instead of a shared, inherited fd), each would have its own offset starting at 0, and their writes **would** overwrite each other's early bytes.

1.8 Stevens' synchronization idiom (fixes the fork ordering race, ties directly to her Exercise on signals below)

Because `fork()` order is genuinely undefined, if parent and child must coordinate ("each tell the other when it's done"), Stevens uses a signal-based rendezvous (`TELL_WAIT`/`TELL_PARENT`/`TELL_CHILD`/`WAIT_PAR` built on `SIGUSR1`/`SIGUSR2`). **This is the direct mechanism her Exercise 2 variant ("write**

alternately from parent and child… using signals") is asking you to implement — see Part 2's exercises.

PART 2 — Signals

2.1 What signals are

Signals are software interrupts — a way of handling asynchronous events in a program. Signals you've already encountered: `SIGINT`, `SIGTERM`, `SIGSEGV`, `SIGBUS`, `SIGCHLD`, `SIGKILL`.

Three possible actions on receiving a signal: 1. **Ignore** it 2. **Catch** it (run a handler) 3. Take the **default action**

2.2 The basic (non-POSIX) interface: `signal()`

```c
typedef void (*sighandler_t)(int);
sighandler_t signal(int signum, sighandler_t handler);
```

`SIG_IGN` is the special handler value meaning "ignore this signal."

Her example program

```c
static void my_handler(int signo);

int main(void) {
    if (signal(SIGUSR1, my_handler) == SIG_ERR) {
        printf("Error installing handler for SIGUSR1");
    } else if (signal(SIGUSR2, my_handler) == SIG_ERR) {
        printf("Error installing handler for SIGUSR2");
    } else if (signal(SIGQUIT, SIG_IGN) == SIG_ERR) {
        printf("Error installing handler for SIGQUIT");
    }
    for (;;) pause();
}

static void my_handler(int signo) {
    if (signo == SIGUSR1)
        printf("Received signal SIGUSR1");
    else if (signo == SIGUSR2)
        printf("Received signal SIGUSR1");   /* note: this line is a typo in the reference — it says SIGUS
    else
        printf("Error: received signal %d\n", signo);
}
```

The `for (;;) pause();` loop exists purely to keep the process alive, suspended, waiting for a signal — without it, the process would run to completion and exit almost immediately, likely before any signal arrives. **This is exactly what her first Signals exercise asks you to explore** (see §2.8).

2.3 Deficiencies of `signal()` — memorize both problems

1. **It helps you catch or ignore a signal, but you cannot block a signal** with `signal()` alone.
2. **A signal can be lost.** Classic broken pattern:

```
int my_sigint_hndlr();
...
signal(SIGINT, my_sigint_hndlr);
...
int my_sigint_hndlr(void) {
    signal(SIGINT, my_sigint_hndlr);   /* re-arm inside the handler */
    ...
}
```

Why this loses signals: on historical ("unreliable") signal implementations, the disposition reverts to `SIG_DFL` the instant the signal is delivered/the handler starts running, and only gets re-armed once the handler explicitly calls `signal()` again *inside itself*. If a **second** occurrence of the same signal arrives in the gap between "handler starts running" and "handler re-registers itself," that second occurrence is delivered using the **default** disposition instead of your handler — for a signal whose default action is termination, that second occurrence can **kill the process** instead of being caught. This unreliability is the entire reason POSIX `sigaction()` exists.

2.4 Sending and waiting for signals

```
int raise(int sig);                          /* send a signal to yourself */
int kill(pid_t pid, int sig);                 /* send a signal to pid (or a group — see below) */
unsigned int alarm(unsigned int seconds);    /* schedule SIGALRM after N seconds */
void abort(void);                             /* raise(SIGABRT) */

int pause(void);                              /* suspend until any signal is caught */
unsigned int sleep(unsigned int seconds);     /* suspend for N seconds (built on alarm+pause historically) */
```

`kill()`'s `pid` argument — four cases (a precise, commonly-tested rule): - `pid > 0` → send to that specific process. - `pid == 0` → send to every process in the caller's **process group**. - `pid < 0` (and $\neq -1$) → send to every process whose process group ID equals `|pid|`. - `pid == -1` → send to every process the caller has permission to signal.

`SIGCHLD`: signal sent **automatically to the parent when a child dies** — this is the OS-level hook that lets a parent avoid polling; it's what a `SIGCHLD` handler + `waitpid(..., WNOHANG)` loop reacts to.

2.5 POSIX signals — the reliable interface

Signal sets — `sigset_t`: a data structure with one bit per signal; each bit set/unset via:

```
int sigemptyset(sigset_t *set);
int sigfillset(sigset_t *set);
int sigaddset(sigset_t *set, int signo);
int sigdelset(sigset_t *set, int signo);
int sigismember(const sigset_t *set, int signo);   /* query */
```

Blocking/unblocking:

```c
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
/* how: SIG_BLOCK, SIG_UNBLOCK, SIG_SETMASK */
```

Catching reliably:

```c
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);

struct sigaction {
    void     (*sa_handler)(int);
    void     (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t sa_mask;
    int      sa_flags;
    void     (*sa_restorer)(void);   /* Linux-specific, not normally set by user code */
};
```

- `sa_mask` — additional signals to **block while this handler is running** (the signal being handled is blocked automatically too, unless `SA_NODEFER` is set).
- `sa_flags` — e.g. `SA_RESTART` (auto-restart interrupted slow syscalls).

Other POSIX signal calls:

```c
int sigpending(sigset_t *set);       /* which blocked signals are currently pending */
int sigsuspend(const sigset_t *mask);/* atomically: install mask, then wait for a signal, then restore old
```

Program 10.12 from APUE (her reference) — implementing `signal()` reliably on top of `sigaction()`

```c
Sigfunc *signal(int signo, Sigfunc *func) {
    struct sigaction act, oact;

    act.sa_handler = func;
    sigemptyset(&act.sa_mask);
    act.sa_flags = 0;

    if (sigaction(signo, &act, &oact) < 0)
        return SIG_ERR;

    return (oact.sa_handler);
}
```

This is worth understanding as a *statement*, not just a code snippet: it shows that a "reliable `signal()`" is just `sigaction()` under the hood with an empty additional mask and no special flags — proving that `sigaction()` subsumes `signal()`'s functionality while fixing its races.

Program 10.15 from APUE (her reference) — block, check pending, then atomically unblock-and-wait

```c
static void sig_quit(int);

int main(void) {
    sigset_t newmask, oldmask, zeromask, pendmask;

    if (signal(SIGQUIT, sig_quit) == SIG_ERR)
        err_sys("cant catch SIGQUIT");
```

```c
    /* Block SIGQUIT and save current signal mask */
    sigemptyset(&newmask);
    sigaddset(&newmask, SIGQUIT);

    if (sigprocmask(SIG_BLOCK, &newmask, &oldmask) < 0)
        err_sys("SIG_BLOCK error");
    sleep(5);

    if (sigpending(&pendmask) < 0)
        err_sys("sigpending error");

    if (sigismember(&pendmask, SIGQUIT))
        printf("\nSIGQUIT pending");

    /* allow all signals and pause */
    if (sigsuspend(&zeromask) != -1)
        printf("sigsuspend error\n");

    /* Reset signal mask which unblocks SIGQUIT */
    if (sigprocmask(SIG_SETMASK, &oldmask, NULL) < 0)
        printf("SIG_SETMASK error\n");

    exit(0);
}

static void sig_quit(int signo) {
    printf("in __FUNCTION__\n");
    return;
}
```

Trace this exactly if asked: for 5 seconds, `SIGQUIT` is **blocked** (not ignored) — if the user sends `SIGQUIT` (Ctrl-\) during that window, it becomes **pending**, not delivered. After the `sleep(5)`, `sigpending()` confirms whether it's sitting pending. `sigsuspend(&zeromask)` then **atomically** installs an *empty* mask (i.e., unblocks everything, including the now-pending `SIGQUIT`) and waits — so the pending `SIGQUIT` is delivered essentially immediately, the handler runs, and `sigsuspend()` returns. Note `zeromask` here is used **uninitialized in this exact reference slide code** (a real program would call `sigemptyset(&zeromask)` first) — worth flagging as a potential "spot the bug" trick question.

2.6 Reentrant functions

Definition (hers): *"Those functions whose behavior is not affected by being interrupted in the middle of the execution and called from another context."*

Her example — is this reentrant?

```c
int my_func(void) {
    static int index = 0;
    index++;
    while (index != 1);
    ...
}
```

Answer: NOT reentrant. `index` is `static` — a single shared variable across all invocations, not a local/stack variable. If a signal handler (or another thread) interrupts `my_func()` after it has already incremented `index` to 1, and then that handler **also calls `my_func()`**, the nested call increments `index` to **2**, and then `while (index != 1)` **spins forever** in the nested call (since `index` is now 2, not 1) — and even after the handler eventually returns, the original outer call resumes with `index` left in a corrupted state (still 2, or whatever the nested call decremented it to, if it decremented at all — in this version it never even gets past the `while`, so it hangs). This is the textbook illustration of why signal handlers must avoid calling non-reentrant functions and must avoid touching shared/static state without protection.

2.7 Signals — her summary

- Signals are asynchronous events that can be caught, ignored, or take the default action.
- Most signals terminate the program unless caught. **A few, such as `SIGCHLD`, are ignored by default.**
- **Non-POSIX signals are unreliable** in nature and **do not allow signals to be blocked.**
- **POSIX signals are reliable** — they allow signals to be caught reliably, blocked, or unblocked-and-suspended without loss of signals.
- It's important to have **reentrant functions** when handling signals, since program execution sequence is hard to predict.

2.8 Her exercises — worked answers

Exercise A: *"In program 10.1, APUE, remove the `for (;;)` statement and run the program multiple times and observe the output. Explain the output."* - Program 10.1 is the `my_handler` install-and-`pause()`-forever example above (§2.2). Removing `for (;;) pause();` means `main()` has nothing left to do after installing the handlers — it falls off the end of `main()` and the process **exits almost immediately**. - **Observed behavior across multiple runs:** the handlers are installed, but unless a signal happens to arrive in the vanishingly small window between installation and process exit, **no signal is ever actually received/printed** — the process exits before anything triggers `my_handler`. If you send a signal manually (`kill -USR1 <pid>`) from another terminal fast enough, you *might* catch it mid-window, but this is a genuine race, and most runs simply exit silently. This demonstrates that **catching a signal requires the process to still be alive/waiting when the signal arrives** — installing a handler alone does nothing if the process has already terminated.

Exercise B: *"Create a process in which a file is opened for RW. Then, fork a new process. Alternately, write from parent and child to the file — e.g., 'From Parent(pid)' and 'From Child(pid)' — using signals."* - This is asking you to implement the **Stevens `TELL_WAIT`-style rendezvous** from Part 1.8: parent writes, signals the child (`kill(child_pid, SIGUSR1)`), then blocks (`pause()`/`sigsuspend()`) waiting for the child's signal back; child does the mirror image. Each process only writes when it's "its turn," guaranteed by the signal handshake rather than by luck — producing strictly alternating output, deterministically, instead of a race.

Exercise C: *"Repeat Ex.2 above as follows: After fork(), exec a new program in child. Remember that a child inherits the open descriptors after exec also. Without re-opening the file, write alternately from parent and child processes."* - Same signal-handshake structure as Exercise B, but the child additionally calls `execl()`/`execv()` right after `fork()`, passing the already-open fd's number as a **stringified argument** (exactly as in the `myexec.c`-style pattern from Part 1.5). Because open fd's **survive `exec()`** by default, the new program image can `atoi()` the passed string back into an `int` and `write()` to it directly — no second `open()` needed. The signal handshake for alternation works identically to Exercise B, just now coordinated with a process that's a *different program* after `exec()`.

PART 3 — Pipes and FIFOs

3.1 Introduction

- **Pipes** are IPC only between **related** processes.
- **FIFOs** (a.k.a. **Named Pipes**) can be used between **unrelated** processes, by virtue of having a name.
- Pipes and FIFOs are **unidirectional**. If both processes need to read and write, **two** pipes/FIFOs are needed.
- **Q (hers): "Do you need IPCs between threads or processes or both?"** — Threads within the same process already share the entire address space (globals, heap), so they don't strictly need a kernel IPC mechanism like a pipe to exchange data — a shared variable protected by a mutex suffices. IPC mechanisms like pipes/FIFOs are specifically for crossing **process** boundaries, where address spaces are separate. (Threads *can* still use pipes if convenient, but it's not required the way it is between processes.)
- Pipes and FIFOs **return file descriptors** and can be treated exactly like files — *"for Unix, the world is a file."*

3.2 IPC persistence — a classification she explicitly tests

Three types of persistence, based on the lifetime of the IPC object: | Persistence type | Lasts until… | Examples | |—|—|—| | **Process-persistent** | the last process with the object open closes it or exits | **Pipes, FIFOs, mutexes, condition variables** | | **Kernel-persistent** | the kernel reboots, or the object is explicitly deleted | named semaphores, shared memory | | **File-persistent** | the object is explicitly deleted | some implementations of named semaphores and shared memory |

This table is a strong candidate for a direct "classify the following IPC mechanism" question.

3.3 Pipes

```
int pipe(int filedes[2]);
```

- Returns two fd's: `filedes[0]` for read, `filedes[1]` for write — **one unidirectional pipe**.
- **For a duplex pipe, two calls to `pipe()`** are made. Then: the **parent closes `fd[0]` of pipe1 and `fd[1]` of pipe2**, while the **child closes `fd[1]` of pipe1 and `fd[0]` of pipe2** — leaving parent with pipe1's write end + pipe2's read end, and child with pipe1's read end + pipe2's write end.
- **The shell's `|` operator is literally the creation of a pipe** between the two processes being run — worth stating plainly if asked "what is `|` implemented with."

Her duplex pipe example program

```
int main(int argc, char **argv) {
    int    pipe1[2], pipe2[2]; pid_t cpid;
    char   str[MAXLEN];

    pipe(pipe1); pipe(pipe2);
    if (fork() == 0) {
        close(pipe1[1]); close(pipe2[0]);
        strcpy(str, "Sending data to parent");
        write(pipe2[1], str, strlen(str));
        read(pipe1[0], str, MAXLEN);
```

```
        printf("%s\n", str);
    } else if (cpid > 0) {
        close(pipe1[0]); close(pipe2[1]);
        strcpy(str, "Sending data to child");
        write(pipe1[1], str, strlen(str));
        read(pipe2[0], str, MAXLEN);
        printf("%s\n", str);
    } else printf("Error forking\n");
    exit(0);
}
```

Bug worth noticing (good trick-question fodder): `cpid` is never assigned the return value of `fork()` — the code does `if (fork() == 0) {...} else if (cpid > 0) {...}`, checking an **uninitialized** `cpid` in the `else if`, rather than the actual parent branch value. In a corrected version, you'd write `pid_t cpid = fork(); if (cpid == 0) {...} else if (cpid > 0) {...}`. If your professor gives you this exact snippet and asks "what's wrong with it," this is the answer.

3.4 FIFOs

```
int mkfifo(const char *pathname, mode_t mode);
int open(const char *pathname, int flags, mode_t mode);
```

- FIFOs need **two calls**: `mkfifo()` to create the named pipe, then `open()` to actually get a usable fd.
- For duplex communication, **two FIFOs** are needed — parent opens FIFO1 for write and FIFO2 for read; child does the opposite.
- **"The sequence of calls for opening the FIFOs is important as it can, otherwise, lead to a deadlock."** — this is the rendezvous-ordering point covered in depth below.

Program 4.16, UNP vol.2 (her reference) — the canonical FIFO client/server

```
#define FIFO1 "/tmp/fifo.1"
#define FIFO2 "/tmp/fifo.2"

void client(int, int), server(int, int);

int main(int argc, char **argv)
{
    int readfd, writefd;
    pid_t childpid;

    /* create two FIFOs; OK if they already exist */
    if ((mkfifo(FIFO1, FILE_MODE) < 0) && (errno != EEXIST))
        printf("can't create %s\n", FIFO1);

    if ((mkfifo(FIFO2, FILE_MODE) < 0) && (errno != EEXIST)) {
        unlink(FIFO1);
        printf("Can't create %s\n", FIFO2);
    }

    if ((childpid = Fork()) == 0) { /* child */
        readfd  = Open(FIFO1, O_RDONLY, 0);
        writefd = Open(FIFO2, O_WRONLY, 0);
```

```
        server(readfd, writefd);
        exit(0);
    }
    /* PARENT */
    writefd = Open(FIFO1, O_WRONLY, 0);
    readfd  = Open(FIFO2, O_RDONLY, 0);
    client(readfd, writefd);
    waitpid(childpid, NULL, 0);
    close(readfd);
    close(writefd);

    unlink(FIFO1);
    unlink(FIFO2);
    exit(0);
}
```

(`Fork()`/`Open()` capitalized here are Stevens' own error-checking wrapper functions from his `unp.h` — functionally identical to `fork()`/`open()` but they `err_sys()`/exit on failure instead of returning -1 silently; don't be thrown if your professor's code uses the capitalized wrapper names.)

Why the crossed order avoids deadlock (the exact mechanism): opening a FIFO for `O_RDONLY` **blocks until a writer opens it**, and opening for `O_WRONLY` **blocks until a reader opens it**. The **child** opens FIFO1 read-first, then FIFO2 write-first. The **parent** opens FIFO1 write-first, then FIFO2 read-first. Child's FIFO1-read pairs with parent's FIFO1-write; parent's FIFO2-read pairs with child's FIFO2-write — every blocking open has a matching counterpart already coming, so both processes eventually proceed. If, instead, **both** processes tried to open the **same** FIFO for the **same** direction first, both would block waiting for a counterpart that will never arrive first (since the counterpart is itself stuck the same way) — **deadlock**.

3.5 Properties of pipes and FIFOs

- A descriptor can be made **non-blocking** with the `O_NONBLOCK` flag in `open()`.
- Two constants govern their operation:
  - **`OPEN_MAX`** — max number of descriptors a process can have open.
  - **`PIPE_BUF`** — max amount of data that can be **atomically** transferred through a pipe/FIFO in one `write()`.
- **Writing to a pipe/FIFO not open for reading (i.e., closed) → `SIGPIPE`, and the process terminates** (by default).
- **Her Q: "No signal is generated when a `read()` is issued from a pipe that has been closed. WHY?"**
  - **Answer:** because that situation is not an error — it's the **normal, well-defined EOF condition**. When all write ends of a pipe are closed, a `read()` on it simply returns **0** (EOF), exactly like reaching the end of a regular file — there's nothing exceptional happening that would warrant interrupting the reader with a signal. `SIGPIPE` exists specifically for the **write** side, where "the intended recipient is gone" genuinely is treated as an error condition worth flagging asynchronously (since a blind `write()` has no other way to immediately notice), whereas a **reader** naturally receives a well-defined return value (0) directly from `read()` itself — no separate out-of-band notification mechanism is needed.

Non-blocking write() behavior — her exact rule table (very precise, easy to mis-state)

When a descriptor is set to non-blocking, `write()`'s behavior depends on the number of bytes you're trying to write vs. `PIPE_BUF` vs. available space:

Condition	Result
`write-bytes < PIPE_BUF` **and** available space < write-bytes	returns **`-1/EAGAIN`** (won't do a partial atomic write below `PIPE_BUF`)
`write-bytes > PIPE_BUF` **and** no available space at all	returns **`-1/EAGAIN`**
`write-bytes > PIPE_BUF` **and** available space < write-bytes (but > 0)	returns the **actual number of bytes written** (a partial write)

The key distinguishing idea: **writes at or below `PIPE_BUF` are all-or-nothing** (atomic) — if they can't fit entirely, you get `EAGAIN`, never a partial write. **Writes above `PIPE_BUF`** are allowed to be **partial** — the kernel will write however much space is currently available and tell you the actual count, rather than failing outright, as long as *some* space exists.

3.6 Pipes and FIFOs — her summary

- Pipes and FIFOs are **unidirectional** IPC mechanisms with **process persistence**. For duplex communication, **two** pipes or FIFOs must be created.
- **Pipes** can be used only between **related** processes.
- **FIFOs** can be used between any two arbitrary processes that know the FIFO's name.
- FIFOs **must be opened in the right order** to avoid deadlock between client and server.
- The number of pipes/FIFOs a process can have open is limited by **`OPEN_MAX`**; the amount of data writable to them (atomically) is limited by **`PIPE_BUF`**.

PART 4 — Mutexes and Condition Variables

4.1 Introduction

Mutexes and condition variables come from the **POSIX thread library (`pthread`)** and are for **synchronization between threads in a process**. They **can** also be used across processes if stored in memory that's **shared between processes** (e.g., via shared memory) — but their "native" habitat is intra-process, inter-thread. - **Mutex** — for **mutual exclusion**. - **Condition variable** — allows a thread to **wait until a condition** becomes true.

4.2 Mutexes

```
pthread_mutex_t fastmutex = PTHREAD_MUTEX_INITIALIZER;
retval = pthread_mutex_lock(&fastmutex);
... Critical Section ...
retval = pthread_mutex_unlock(&fastmutex);
```

Primary functions: lock and unlock, such that **only one thread is allowed to acquire the lock** at a time.

Her producer/consumer-with-mutex example

```
#define MAXITEMS   100
#define MAXTHREADS 10

int nitems;

struct share_s {
    pthread_mutex_t mutex;
    int             buff[MAXITEMS];
    int             nput;
    int             nval;
} shared = {PTHREAD_MUTEX_INITIALIZER};

int main(int argc, char **argv)
{
    int i, nthr;
    pthread_t prod[MAXTHREADS], cons;

    nthr = atoi(argv[1]);
    nitems = nthr;
    shared.nput = shared.nval = 0;

    for (i = 0; i < nthr; i++)
        pthread_create(&prod[i], NULL, produce, &i);
    for (i = 0; i < nthr; i++)
        pthread_join(prod[i], NULL);

    pthread_create(&cons, NULL, consume, NULL);
    pthread_join(cons, NULL);

    exit(0);
}

static void *produce(void *arg)
{
    pthread_mutex_lock(&shared.mutex);
    if (shared.nput >= nitems) {
        pthread_mutex_unlock(&shared.mutex);
        return NULL;
    }
    shared.buff[shared.nput] = shared.nval;
    shared.nput++;
    shared.nval += 10;
    pthread_mutex_unlock(&shared.mutex);
    return NULL;
}

static void *consume(void *arg)
{
    int i;
    for (i = 0; i < nitems; i++)
        printf("buff[%d] = %d\n", i, shared.buff[i]);
    return NULL;
```

```
}
```

Read this program carefully — it has a real design point worth noting: all producer threads are `pthread_join()`ed **before** the consumer thread is even created. So this particular program does **not** actually demonstrate concurrent producer/consumer overlap — it demonstrates **mutual exclusion among the producers themselves** (guarding `shared.nput/shared.nval` while multiple producer threads race to increment them), with consumption happening safely afterward, sequentially, with no concurrent access needed at that point. **This is a good "what does this program actually demonstrate" trick question** — it's easy to assume it's a live producer-consumer pipeline when it's structurally not (that's what condition variables are for, in §4.3 below).

4.3 Condition variables

- Allow a thread to **wait for a condition**.
- A condition variable **always has a mutex associated with it**, since mutual exclusion is needed to safely check/modify the condition itself.
- Either a **single** thread or **multiple** threads can be woken by one condition-variable signal.

```
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_cond_broadcast(pthread_cond_t *cond);
```

Her canonical code pattern

```
SIGNAL-PROCESS:
  pthread_mutex_lock(&var.mutex);
  set condition true
  pthread_cond_signal(&var.cond);
  pthread_mutex_unlock(&var.mutex);

WAIT-PROCESS:
  pthread_mutex_lock(&var.mutex);
  while (condition is FALSE)
      pthread_cond_wait(&var.cond, &var.mutex);
      modify condition
  pthread_mutex_unlock(&var.mutex);
```

Two details worth being precise about, since this differs slightly in emphasis from a generic textbook version: - The **`while`, not `if`**, guarding `pthread_cond_wait()` is essential — exactly as in Stevens/POSIX generally — to protect against spurious wakeups and multiple waiters. - Her pseudocode shows **"modify condition"** positioned *after* the `while` loop (i.e., after being woken and confirming the condition is now true) — meaning the **waiter itself** may adjust the condition variable's underlying state (e.g., decrementing a count) once it wakes and proceeds, not just the signaler. This is a natural pattern (think: a shared counter where waking threads each consume one unit), and it's worth being ready to explain *why* this "modify" step must still happen **while holding the mutex** (it does, since it's before `pthread_mutex_unlock()`) — otherwise you'd reintroduce the exact race condition variables exist to prevent.

4.4 `pthread_mutex_lock()` internals worth stating precisely

- `pthread_mutex_lock(&mutex)` **blocks** the calling thread if another thread currently holds the mutex, and returns once it successfully acquires the lock.
- `pthread_cond_wait()`'s unlock-then-block is **atomic** with respect to other threads trying to acquire that same mutex to signal the condition — this atomicity is what prevents a "lost wakeup": if unlocking and blocking were two separate, non-atomic steps, a signaling thread could acquire the mutex and call `pthread_cond_signal()` in the gap between the waiter's unlock and its actual blocking, and that signal would be missed entirely.

4.5 Her exercises for Mutexes and Condition Variables — worked answers

Exercise A: *"Create threads in a process and put them to sleep and after waking up, print their thread ID and die. Let the main thread wait for the child tid status. What is the order in which they print? Try multiple times."* - Each thread calls `sleep()` for some duration then prints its own `pthread_self()` and exits; main calls `pthread_join()` on each in turn. **The print order across runs is non-deterministic** — it depends on each thread's actual sleep duration and OS scheduling, exactly analogous to the `fork()` ordering race in Part 1, just at the thread level instead of the process level. If all threads sleep for the *same* fixed duration, the order is still not guaranteed identical run-to-run, since scheduling and wakeup latency vary.

Exercise B: *"Repeat the above after removing the `pthread_wait()` [i.e., `pthread_join()`] from the main thread. What is the output?"* - Without joining, **`main()` may reach the end of the program (and call `exit()`, terminating the whole process, all threads included) before some or all of the worker threads have woken up and printed anything.** Result: some or all of the expected thread output can be **missing entirely**, non-deterministically — this is the multi-threaded analogue of "the process exited before the signal arrived" from the Signals Exercise A in Part 2.8. It underlines *why* `pthread_join()` (or an equivalent explicit synchronization) is necessary if you actually need to observe every thread's work complete.

Exercise C: *"Repeat the above such that a global variable is updated by the threads. The global variable starts with value 0 and when it is 0, the main thread prints its thread ID and increments it. Then, thread 1 must wake up, print its thread ID, increment the global variable and die and so on. The main thread waits for the status of all threads before it dies."* - This is a **condition-variable "ticket"/turn-taking pattern**: a shared `int turn = 0;` guarded by a mutex+condition variable. Each participant (main thread first, then thread 1, thread 2, ...) does: `lock → while (turn != my_expected_value) cond_wait(...) → print my ID → turn++ → cond_broadcast(...) (wake everyone to re-check) → unlock`. Using `pthread_cond_broadcast()` rather than `pthread_cond_signal()` is the safer choice here, since multiple threads may be waiting on different target `turn` values simultaneously and each needs the chance to re-check its own condition after every increment. This exercise is essentially forcing you to implement **strict ordering among threads using a condition variable as a turnstile** — a very natural follow-up exam question is "write the code for this."

4.6 Mutexes and Condition Variables — her summary

- **Mutexes** are for mutual exclusion — acquire lock, update global variables, then unlock. Only one thread executes the critical section at a time.
- **Condition variables** are needed for waiting on a condition. A condition variable is always associated with a mutex, since it's accessed by multiple threads that must access it mutually exclusively.

PART 5 — Tricky concepts & likely questions, by topic

5.1 Process Control — tricky points

- **Trap:** don't confuse "copy-on-write" with "the child doesn't get its own memory." COW is purely a *lazy-copy optimization*; semantically the child still behaves as if fully independent from the instant of `fork()`.
- **Trap:** Program 8.1's `write()` before `fork()` is unbuffered and prints once, guaranteed — but the `printf("before fork\n")` right after it can print **once or twice** depending on buffering mode. If asked to explain *both* lines in the same program, don't apply the same reasoning to both — they behave differently.
- **Trap:** in Program 8.1, the **parent sleeps, the child does not** — this is the *opposite* of what students often assume ("parent runs first"); it's specifically engineered so the child's output is very likely to interleave predictably relative to the parent's *final* print, though the two print statements before the sleep/increment are still each independently subject to the general fork race.
- **Q she might ask directly:** "Why does `exec()`'s inherited-properties list include file locks and pending signals, but not the program's own data/heap/stack?" — because `exec()`'s entire purpose is replacing the program image (text/data/heap/stack); everything **not** part of "the program's own code and memory" — process-identity and kernel-tracked state like pid, locks, pending signals, cwd — has no reason to be reset, since none of that is "part of the old program."
- **Q:** "A concurrent server forks a child per connection. Under sustained load, it starts failing to accept new connections even though the machine has free CPU and memory. Why?" — Answer: unreaped **zombies** are exhausting `OPEN_MAX`/process-table limits (per-user or per-system process count), not CPU/memory — this is precisely the professor's own warning in her Process Control summary.

5.2 Signals — tricky points

- **Trap:** the "lost signal" example in §2.3 is subtle — students often think the danger is "the handler doesn't get called," but the actual danger is the **opposite**: a *second* occurrence in the re-arming gap gets the **default action** (often termination), not silence. Be precise about which failure mode occurs.
- **Trap:** `SIGCHLD`'s default action is to be **ignored**, not to terminate — one of the few signals where "ignored" is the *default* disposition, not something you set explicitly. But note: default-ignored is different from explicitly setting `SIG_IGN` — explicitly setting `SIG_IGN` on `SIGCHLD` specifically triggers **automatic zombie reaping** on POSIX-compliant systems, a behavior the *default* ignore-disposition does not necessarily give you the same guarantee about across all historical systems (this nuance is a favorite "gotcha" distinction).
- **Trap (reentrancy example):** the failure mode in §2.6 isn't just "wrong value" — it's an actual **infinite spin/hang** (`while (index != 1);` with no way out once `index` becomes anything other than 1 permanently), which is a stronger and more dramatic failure than students usually expect from "just a race condition."
- **Q:** "In Program 10.15, what would happen if `sigaddset(&newmask, SIGQUIT)` were never called (i.e., `newmask` stayed empty)?" — `sigprocmask(SIG_BLOCK, &newmask, &oldmask)` would then block **nothing**, so `SIGQUIT` sent during the `sleep(5)` would be delivered **immediately** (handled by `sig_quit()` right away) rather than sitting pending — `sigpending()` afterward would show it as **not** pending (since it was already handled), changing the entire trace of the program.
- **Q:** "Why does `sigsuspend()` need to be atomic, specifically?" — because "unblock, then wait" as two separate calls leaves a window where a signal arriving in between would simply be delivered using whatever the *default* pending/blocked state was at that exact instant, potentially getting lost relative to your intended wait — same "lost wakeup"

category of bug as the mutex/cond-var atomicity requirement in Part 4.

5.3 Pipes/FIFOs — tricky points

- **Trap:** many students assume `SIGPIPE` fires on `read()` from a pipe with no writer, by symmetry with the write case. **It does not** — that's a completely normal EOF (`read()` returns 0), not an error. Only the **write** side generates `SIGPIPE`, and only when there's no reader. Her slide question exists specifically to catch this asymmetry.
- **Trap:** the non-blocking `write()` table (§3.5) is often mis-remembered as "small writes always succeed partially" — they don't; **writes at or below `PIPE_BUF` are strictly all-or-nothing.** Only writes **above** `PIPE_BUF` can return a partial byte count.
- **Trap:** the duplex pipe program in §3.3 has a real bug (uninitialized `cpid` used in the `else if`) — if she hands you this exact code and asks "will this compile/run correctly," the honest answer is it likely still "works" by accident on most compilers (since the `else if` branch is only reached when `fork()` did **not** return 0, and an uninitialized local's value is usually nonzero garbage, statistically probably read as ">0"), but it's **undefined behavior**, not something you should rely on — this is exactly the kind of code-reading trap she seems to enjoy testing.
- **Q:** "Why do FIFOs need the open-order convention but pipes don't?" — because pipe fd's are handed to both processes directly via `fork()` inheritance (no `open()` rendezvous involved at all — the fd's already exist), whereas each process using a FIFO must independently call `open()` on a named path, and it's *that* independent open() call that has blocking rendezvous semantics — a distinction that doesn't exist for pipes.

5.4 Mutexes/Condition Variables — tricky points

- **Trap:** the producer/consumer mutex program in §4.2 is *not* actually demonstrating live concurrent producer-consumer interleaving — all producers finish (joined) before the consumer even starts. If asked "does this program need a condition variable," the honest answer is **no, not as written** — a condition variable would only become necessary if the consumer needed to start consuming *while* production was still ongoing (i.e., truly interleaved), which this particular structure avoids by fully serializing the two phases via `pthread_join()`.
- **Trap:** "modify condition" appearing *after* the `while (condition is FALSE) pthread_cond_wait(...)` line in her pseudocode is easy to misplace — it must stay **before** `pthread_mutex_unlock()`, i.e., still inside the critical section, or you reintroduce a race on the very state you just waited to safely observe.
- **Q:** "Why must a condition variable always have an associated mutex — could you implement 'wait for a condition' with just a mutex and a busy-loop instead?" — Technically you could spin-check a flag under repeated lock/unlock, but that **busy-waits** (wastes CPU, and depending on your lock, can also **starve** the signaling thread of the lock it needs to actually flip the flag) — `pthread_cond_wait()` avoids busy-waiting entirely by having the OS **not schedule the waiting thread at all** until signaled, which is the whole point of a condition variable over manual polling.

PART 6 — Consolidated Question Bank (75+ questions)

Process Control

1. What process creates every other process in UNIX (besides `init`)?
2. State the copy-on-write rule for `fork()` precisely, as covered in lecture.

3. In Program 8.1, why does the initial `write()` call print exactly once regardless of buffering, while the following `printf()` might print once or twice?
4. In Program 8.1, why does the parent sleep for 2 seconds — what race does this avoid?
5. List everything a child inherits from its parent via `fork()`, per the lecture slide.
6. What's different between parent and child immediately after `fork()`?
7. State the file-sharing rule for a fd shared across `fork()`, and explain why writes from both processes don't overwrite each other.
8. What happens if, instead of sharing an inherited fd, parent and child each independently `open()` the same file and then write? Contrast with Q7.
9. Distinguish normal vs. abnormal process termination, and name the calls involved.
10. What happens when a parent dies before its child? When a child dies before its parent (unreaped)?
11. Why are zombies a genuine operational risk for a concurrent server?
12. What properties are inherited across `exec()`, per the lecture's exact list?
13. Why do file descriptors (though not explicitly listed) still survive `exec()`?
14. Explain the three kernel data structures behind file sharing, and how they explain Q7.
15. What mechanism (from Stevens) lets a parent and child deterministically alternate/synchronize instead of racing?

Signals

16. Define a signal, per the lecture's own phrasing.
17. What are the three possible actions on receiving a signal?
18. What are the two deficiencies of `signal()` listed in lecture?
19. Trace through the "lost signal" re-arming example — what specifically can go wrong, and why?
20. What are the four cases of `kill()`'s `pid` argument?
21. What is `SIGCHLD` sent for, and by whom?
22. What is a `sigset_t`, and name the five functions used to manipulate/query one.
23. What does `sigprocmask()`'s `how` argument control, and what are its three values?
24. List the fields of `struct sigaction` as given in lecture.
25. Walk through Program 10.15 exactly — what is printed, and in what order, assuming `SIGQUIT` is sent during the 5-second sleep?
26. What does `sigpending()` tell you that `sigprocmask()` alone cannot?
27. Why must `sigsuspend()`'s mask-install-and-wait be atomic?
28. Define a reentrant function, per the lecture's own phrasing.
29. Trace the `static int index` example — why does it hang rather than just producing a wrong value?
30. What's the default disposition of `SIGCHLD`, and how does that differ from explicitly setting `SIG_IGN` on it?
31. Why does removing `for (;;) pause();` from Program 10.1 change its observed behavior?
32. Why are non-POSIX signals called "unreliable," specifically?

Pipes and FIFOs

33. Why are pipes restricted to related processes, but FIFOs are not?
34. Why are pipes/FIFOs described as unidirectional, and what's needed for duplex communication?
35. Do threads strictly need pipes/FIFOs for IPC? Why or why not?
36. Classify pipes, FIFOs, mutexes, and condition variables by persistence type (process/kernel/file).
37. What do `fd[0]` and `fd[1]` mean after `pipe()`?
38. For a duplex pipe setup, exactly which fd's does the parent close, and which does the child close?

39. What is `|` in the shell implemented with?
40. Spot the bug in the duplex pipe example program (§3.3) — what's wrong with how `cpid` is used?
41. What two calls are needed to get a usable FIFO fd, and why two?
42. Why must FIFO open-order be "crossed" between client and server, and what happens if it isn't?
43. What are `OPEN_MAX` and `PIPE_BUF`, precisely?
44. What happens when you write to a pipe/FIFO with no reader?
45. Her Q: why does no signal fire when you `read()` from a pipe that's been closed (no writer)?
46. State the exact non-blocking `write()` behavior table (three cases) for writes vs. `PIPE_BUF` vs. available space.
47. Why are sub-`PIPE_BUF` writes never partial, while above-`PIPE_BUF` writes can be?

Mutexes and Condition Variables

48. What library are mutexes and condition variables part of, and what's their "native" use case (threads vs. processes)?
49. Under what condition can mutexes/condition variables be used across processes rather than just threads?
50. What is a mutex for? What is a condition variable for?
51. Walk through the producer/consumer mutex example (§4.2) — does it demonstrate live concurrent producer-consumer overlap? Justify your answer.
52. Why does a condition variable always need an associated mutex?
53. Can more than one thread be woken by a single condition-variable operation? Which call does that, specifically?
54. Write out the SIGNAL-PROCESS / WAIT-PROCESS pseudocode pattern for condition variables from memory.
55. Why must the "modify condition" step happen before `pthread_mutex_unlock()`, not after?
56. Why must `cond_wait()` be inside a `while`, not an `if`?
57. Why is `pthread_cond_wait()`'s unlock-then-block required to be atomic?
58. In the sleeping-threads exercise, why is the print order non-deterministic across runs?
59. In the same exercise, what happens if you remove `pthread_join()` from main — why can thread output go missing?
60. Design (in words, or in code) the "turnstile" condition-variable pattern for the strict-ordering exercise (§4.5, Exercise C). Why is `pthread_cond_broadcast()` the safer choice there over `pthread_cond_signal()`?

Cross-topic / integrative

61. Compare the `fork()` ordering race (Part 1) to the `alarm()`/`pause()` race and the sleeping-threads race (Part 4) — what do all three have in common structurally?
62. Compare how a pipe and a mutex+condition-variable pair each implement "block until something is ready" — what's fundamentally different about the mechanism (kernel-buffered I/O vs. explicit application-level synchronization)?
63. Both signals and condition variables have a "lost wakeup"/"lost signal" failure mode. Name the specific fix in each case (signals: what interface; condition variables: what atomicity guarantee).
64. Why does zombie prevention (Part 1) and reentrancy (Part 2) both ultimately trace back to the same root cause — unpredictable execution ordering?
65. A concurrent server (Part 1's TCP Concurrent Server pattern) forks a child per connection and never installs a `SIGCHLD` handler. Diagnose the eventual failure mode using concepts from Parts 1 and 2 together.

Final tip: for anything drawn directly from a named/numbered Stevens program (8.1, 10.1, 10.12, 10.15, UNP 4.16), be ready to reproduce the program's structure from memory, not just recall its conclusion — she has shown a clear pattern of testing via "trace this exact code" rather than only abstract definitions.